

Embedding Trustworthy AI into Quantitative Systems Engineering Tools

Ray Madachy
Naval Postgraduate School
Systems Engineering Dept.
rjmadach@nps.edu

Background

This presentation is based on the following INCOSE INSIGHT special issue article with additions:

R. Madachy. Trustworthy AI for Quantitative Systems Engineering via Deterministic Delegation to Verified Tools. INCOSE INSIGHT, June 2026.
https://incose.onlinelibrary.wiley.com/doi/10.1002/inst.70056

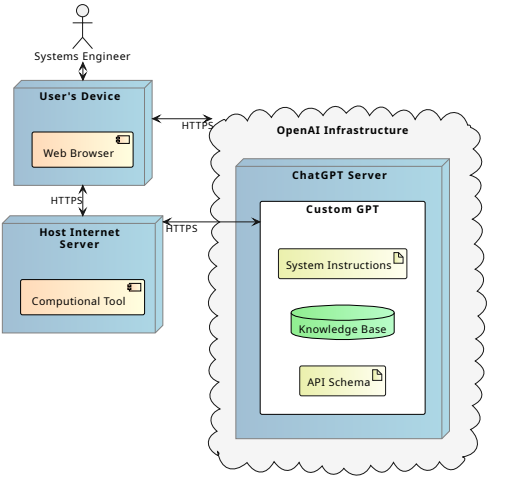


Approach

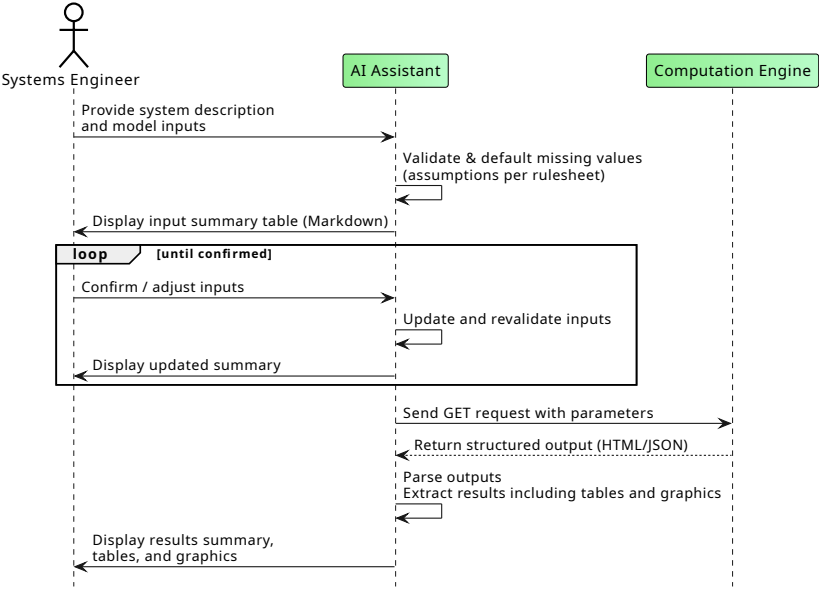
- Restrict the LLM from authoritative mathematical computation when a verified quantitative tool is available.
- Delegate quantitative model calculations to independently validated external tools through formally defined APIs.
- Enforce structured invocation with typed parameters, named operations, and constrained endpoints.
- Use multi-modal Generative AI for smart input inference from natural language, documents, and images, with explicit assumptions and user review.
- Return structured outputs with provenance, including interpreted inputs, assumptions, model/tool results, and source URLs for independent verification and rerun.

General Cloud-based Deployment Architecture

- Distributed execution environments for user interaction, AI mediation, and independently hosted computational services.
- LLM governed by system instructions, domain knowledge base, and formally defined API schema.
- All authoritative computation delegated to independently validated external tools; the LLM interprets, orchestrates, and presents results.
- Each domain adapts the knowledge, instructions, API schema, and backend service while retaining the same delegation pattern [1].



Operational Sequence Model Diagram



Custom GPT Design

- Demonstrated assistants built using the OpenAI Custom GPT framework with explicit instructions, domain knowledge, and API schemas. The delegation pattern itself is platform independent.
- Configured with deterministic delegation rules that prohibit authoritative internal computation when the approved external tool is available.
- Inputs gathered from users in natural language, uploaded documents, or pasted visual media. Missing values may be resolved through clarification or explicit assumptions.
- Reliability assistants reduce requests to reliability parameters and structured RBD models. The cost modeling assistant translates project descriptions to size and cost driver inputs for the Constructive Systems Engineering Cost Model (COSYSMO) [2].
- Outputs preserve quantitative results, input rationale and assumptions, tables, graphics, and calculation source links for verification and parameter modification.

Assistant Instructions

- Instructions constrain assistant behavior via elicitation rules, permissible assumptions and defaults, required tool usage, and output formatting. They govern behavior beyond the formal API contract.
- Quantitative requests are explicitly delegated to the appropriate deterministic computational tool rather than computed by the LLM.
- Typically written in Markdown to support hierarchical organization, readability by both humans and machines, and structured operational logic.
- Domain-specific instructions reduce natural language requests into model parameters required by the corresponding tool.
- The same delegation pattern is applied across different quantitative systems engineering applications.

Assistant Instruction Excerpts

Parallel Component Reliability GPT

```
## Processing
Reduce reliability requests to:
- mtbf
- mission_time
- redundancy

State reasonable assumptions. Ask for required
parameters that cannot be confidently estimated.

Use the named schema operation `ReliabilityCalculate`
sending the following parameters. Do not compute internally.

```json
{
 "mtbf": null,
 "mission_time": null,
 "redundancy": null
}
```
Outputs
After the tool returns, summarize:
- component reliability
- system reliability
- system failure probability
Also include:
- the exact calculation source URL
- the returned fault tree rendered in a fenced `svg` block
```

COSYSMO GPT

```
## Core Rule
Never compute COSYSMO results internally.
All quantitative model calculations MUST be
delegated to the external COSYSMO tool via
the `estimateCosysmo` API operation.

## Input Collection
Elicit system size drivers at Easy / Nominal /
Difficult levels. Collect all 14 cost driver
ratings. Apply defaults: missing size fields = 0;
unspecified cost factors = Nominal.

## Tool Invocation
Invoke `estimateCosysmo` with all validated
parameters. Parse returned effort, schedule,
cost, total size, effort distribution, and
graphics.

## Provenance
Append the exact calculation source URL so
the user can independently verify, modify,
and rerun the estimate.
```

Local LLM Instruction Excerpt

```
SYSTEM_PROMPT = r"""
You are COSYSMO Estimator, a systems engineering cost estimation
assistant.

You are the natural-language front end to the deterministic
COSYSMO calculation tool: estimateCosysmo

ABSOLUTE CALCULATION RULE

If the user asks for effort, Person-Months, schedule, cost,
equivalent nominal size, an estimate, recalculation, COSYSMO
calculation, or the result of changing a COSYSMO parameter,
you MUST call estimateCosysmo.

Never answer a COSYSMO calculation request from memory,
approximation, rules of thumb, ranges, guesses, your own
arithmetic, or prior training data.

The deterministic estimateCosysmo tool is the sole authority for
calculated COSYSMO values.

EXAMPLE

User:

    What is effort for 100 system requirements?

Required behavior:

    Call estimateCosysmo with:
        System_Requirements_Nominal = 100

Do not estimate effort yourself.
```

```
SYSTEM SIZE

COSYSMO has four system size drivers rated by difficulty:
System Requirements:
    System_Requirements_Easy
    System_Requirements_Nominal
    System_Requirements_Difficult

Interfaces:
    Interfaces_Easy
    Interfaces_Nominal
    Interfaces_Difficult

Critical Algorithms:
    Critical_Algorithms_Easy
    Critical_Algorithms_Nominal
    Critical_Algorithms_Difficult

Operational Scenarios:
    Operational_Scenarios_Easy
    Operational_Scenarios_Nominal
    Operational_Scenarios_Difficult

If the user gives an unqualified size count, interpret it as
Nominal difficulty. Examples are:

    100 system requirements
        -> System_Requirements_Nominal = 100
    25 interfaces
        -> Interfaces_Nominal = 25

Unspecified system size fields are set deterministically to zero
by the Python host.
...
```

API Schemas

- OpenAPI specification in YAML format defines the formal interface contract between GPTs and external tools.
- APIs specify server endpoint, named operation, typed parameters, and expected response format.
- API operations have named IDs which are explicitly referred to in assistant instructions.
- Parameter constraints (e.g., enumerated rating levels) enforce valid inputs before the API call is made.
- API structures are also used to directly call LLMs for individual prompts or full conversations with history.

API Schema Excerpts

Parallel Component Reliability GPT

```yaml
openapi: 3.1.0
servers:
  - url: https://se-lib.org/online/
paths:
  /reliability.py:
    get:
      operationId: ReliabilityCalculate
      parameters:
        - in: query
          name: mtbf
          description: Mean Time Between Failures in hours.
          required: true
          schema:
            type: number
            minimum: 0
        - in: query
          name: mission_time
          description: Mission time in hours
          required: true
          schema:
            type: number
            minimum: 0
        - in: query
          name: redundancy
          description: the number of identical components
          ↪  operating in parallel (integer  1)
          required: true
          schema:
            type: number
            minimum: 1
```

COSYSMO GPT

```yaml
openapi: 3.1.0
info:
  title: COSYSMO Estimation API
  version: 1.0.0
servers:
  - url: https://softwarecost.org/tools/
paths:
  /cosysmo_ai:
    get:
      operationId: estimateCosysmo
      summary: Estimate cost, schedule, and effort using the
      ↪  COSYSMO model
      parameters:
        - in: query
          name: System_Requirements_Easy
          schema: { type: number }
        - in: query
          name: System_Requirements_Nominal
          schema: { type: number }
...
        - in: query
          name: Tool_Support
          schema: { type: string, enum: [Very_Low, Low, Nominal,
          ↪  High, Very_High] }

      responses:
        '200':
          description: HTML with COSYSMO estimation results
          content:
            text/html:
              schema:
                type: string
```

Local LLM API Schema


```
# ============================================================
# 7. LM STUDIO TOOL SCHEMA
# ============================================================

def build_cosysmo_tool():
    properties = {}

    for field in SIZE_FIELDS:
        properties[field] = {
            "type": "number",
            "minimum": 0,
            "description": (
                f"Number of {SIZE_FIELD_LABELS[field]}. "
                "If the user gives a count without specifying
                ↪  Easy/Nominal/Difficult, use Nominal."
            ),
        }

    for field, allowed_values in COST_FACTOR_ENUMS.items():
        properties[field] = {
            "type": "string",
            "enum": allowed_values,
            "description": f"COSYSMO cost driver:
            ↪  {COST_FACTOR_LABELS[field]}.",
        }

    properties["system_labor_cost_per_PM"] = {
        "type": "number",
        "minimum": 0,
        "description": "Systems engineering labor cost in USD
        ↪  per Person-Month. Omit if unspecified; Python
        ↪  defaults it to 20000.",
    }

    properties["calculated"] = {
        "type": "string",
        "enum": ["True"],
        "description": 'COSYSMO calculation flag. Python always
        ↪  sends "True".',
    }

    return {
        "type": "function",
        "function": {
            "name": "estimateCosysmo",
            "description": (
                "Run the authoritative deterministic COSYSMO
                ↪  systems engineering estimation model. "
                "Use for effort, cost, schedule, equivalent
                ↪  nominal size, COSYSMO estimates, and
                ↪  scenario changes."
            ),
            "parameters": {
                "type": "object",
                "properties": properties,
                "additionalProperties": False,
            },
        }
    }

tools = [build_cosysmo_tool()]
```

Applications

System Reliability Analysis Examples

- Reliability analysis is well suited to deterministic delegation because it requires exact probabilistic computation, explicit structural assumptions, and reproducible results.
- Generative AI interprets system descriptions, organizes model structure, identifies missing parameters, documents assumptions, and explains results.
- Validated reliability tools perform the authoritative calculations and generate returned fault-tree or reliability block diagram visualizations.
- Examples progress from direct parallel-redundancy calculation to AI-assisted construction of a structured reliability block diagram from a narrative mission description.

Reliability Example 1: Parallel Redundancy

- Example demonstrates direct deterministic evaluation of a simple parallel system using the Parallel Component Reliability Analyzer GPT [3] calling the deterministic Parallel Component Reliability Analyzer tool [4] which also renders a fault tree.
- For a component with constant failure rate, reliability over mission time is given by:

$$\mathrm{R}_i(t) = e^{-t/\theta i}$$

Where

R_i	is the reliability of the ith component
t	is the mission time (hours)
θ_i	is the MTBF of the ith component as the mean time between failures (hours)

For m identical components operating in parallel, the overall system reliability is:

$$\mathrm{R}_{sys}(t) = 1 - (1 - R_i(t))^m$$

Where

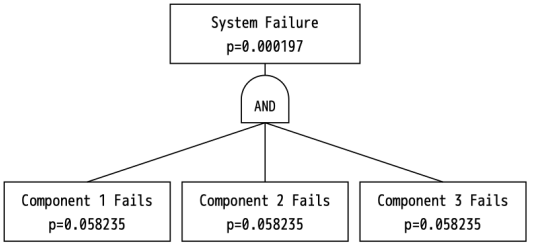
R_{sys}	is the system reliability
R_i	is the reliability of the ith component
m	is the number of identical components

Reliability Prompt and AI Output

Estimate the 5 day reliability of a triple-redundant marine power plant built from Caterpillar diesel generator sets.

Assuming Caterpillar-class industrial diesel generator sets with a default MTBF of 2,000 hours, a 120-hour (5-day) mission, and three active-redundant generators operating in parallel the system reliability is 99.98%. The 2,000 hour MTBF is a conservative engineering estimate derived from publicly available emergency diesel generator reliability studies and adjusted to reflect marine operating conditions.

- Component reliability: 0.941765 (94.18%)
- System reliability: 0.999803 (99.98%)
- System failure probability: 0.000197 (0.02%)



Calculation source ↗

Reliability Calculator with AI Inputs

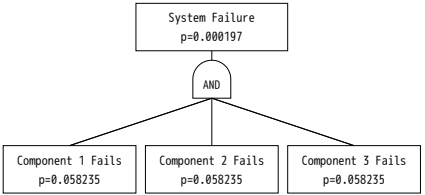
Parallel Component Reliability Calculator

This tool computes system reliability using an exponential (constant failure rate) model. Component reliability is $R = e^{-t / \mathrm{MTBF}}$. Optional redundancy assumes m identical components in parallel: $R_{\mathrm{system}} = 1 - (1 - R_{\mathrm{component}})^{m}$.

MTBF (hours): 2000
Mission time t (hours): 120
Redundancy (parallel $m \geq 1$): 3

Calculate

Results
Component reliability $R_{\mathrm{component}} = 0.941765$ (94.18%)
System reliability $R_{\mathrm{system}} = 0.999803$ (99.98%)
System failure probability $F_{\mathrm{system}} = 0.000197$ (0.02%)



Reliability Example 2: Reliability Block Diagram

- The assistant Reliability Block Diagram Expert GPT [5] translates a narrative into a structured series-parallel RBD and submits appropriate model parameters to the backend Reliability Block Diagram Analyzer tool [6].
- Tool calculates system reliability per the following and renders an RBD:

$$R_{sys}(t) = f(\tau) \quad \text{where} \quad \begin{cases} e^{-t/\theta i} \text{ or } r_i & \tau = \text{Block}_i \\ \prod_{i=1}^{N} f(\tau) & \tau = \text{Series} \\ 1 - \prod_{i=1}^{N} (1 - f(\tau)) & \tau = \text{Parallel} \end{cases}$$

Where

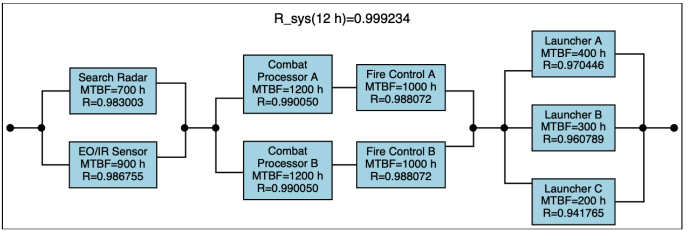
$R_{sys}(t)$	is the overall system reliability at mission time t
$f(\tau)$	is the recursive reliability evaluation function applied to node τ
τ	represents either a component block, series structure, or parallel structure within the RBD
τ_i	is the i-th child node of structure τ
r_i	is the mean time between failures (MTBF) of component i
N	is a directly specified reliability value for component is the number of child nodes within the series or parallel structure

RBD Prompt and AI Output

Create and evaluate a reliability block diagram for a combat system mission that lasts 12 hours. Detection should succeed if either the search radar or the EO/IR sensor is available. The engagement control function should be modeled with two redundant processing paths: Combat Processor A in series with Fire Control A, and the backup path is Combat Processor B in series with Fire Control B. The weapon-launch function should succeed if either Launcher A or Launcher B or Launcher C is available. Use MTBF values of 700 hours for Search Radar, 900 for EO/IR Sensor, 1200 for each Combat Processor, 1000 for each Fire Control component, and the Launchers with respective MTBFs of 400, 300 and 200 hours.

I modeled the combat system as a three-stage series RBD for a 12-hour mission: a redundant detection group, a redundant command-and-engagement-control group with two parallel processing paths, and a three-way redundant launcher group. Using the specified MTBF values and assuming independent failures within each redundancy group, the backend tool computed an overall mission reliability of Rsys(12h)=0.999234. The corresponding mission failure probability is 0.000766.

This result indicates a highly resilient architecture for the stated mission duration. The high mission reliability is driven by redundancy at each major functional stage, which substantially reduces vulnerability to any single component failure. The diagram below shows the exact series-parallel structure evaluated by the backend reliability tool and the component-level reliabilities used in the calculation.



Calculation source ↗

RBD Tool with AI Inputs

Reliability Block Diagram Generator

Enter the RBD using Series(...), Parallel(...), and Block(...). Use MTBF values in hours.

Diagram title

```
Combat System Reliability Block Diagram
```

Mission time (hours)

```
12
```

Output filename stem

```
rbd_output
```

RBD model

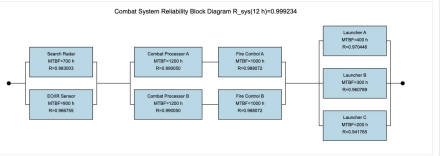
```
Series([
  Parallel([
    Block("Search Radar",700),
    Block("EO/IR Sensor",900)
  ]),
  Parallel([
    Series([
      Block("Combat Processor A",1200),
      Block("Fire Control A",1000)]),
    Series([
      Block("Combat Processor B",1200),
      Block("Fire Control B",1000)])
  ]),
  Parallel([
    Block("Launcher A",400),
    Block("Launcher B",300),
```

Generate RBD

RBD Results

System reliability R_{sys} = 0.999234 (99.9234%)

System failure probability Q_{sys} = 0.000766 (0.0766%)



Systems Engineering Cost Modeling Application

- Cost modeling combines formal quantitative estimation with incomplete or ambiguous project information.
- Generative AI bridges ordinary project descriptions to complete model parameter sets through elicitation, normalization, documented assumptions, and reviewable inputs.
- The quantitative calculation remains outside the LLM and is delegated to the verified COSYSMO implementation.
- Demonstrated with the COSYSMO GPT [7] calling the COSYSMO with AI Usage tool [8], and local Python script with LM Studio calling desktop tool.

COSYSMO Effort Equation

$$Effort = A \cdot \left(\sum_{k} (w_e \Phi_{e,k} + w_n \Phi_{n,k} + w_d \Phi_{d,k}) \right)^{B} \cdot \prod_{j=1}^{14} EM_j$$

Where

- Effort is in Person-Months (PM) for a nominal schedule
- A is a constant derived from historical project data
- k = Numbers of Requirements, Interfaces, Algorithms and Scenarios
- w_x = weight for easy (e), nominal (n), or difficult (d) size driver
- Φ is the quantity of "k" size driver
- B is an exponent for the diseconomy of scale
- EM_i is an effort multiplier for the ith cost driver. The geometric product is an overall Effort Adjustment Factor (EAF) to the nominal effort.

Example Cost Estimate 1

What is the estimated systems engineering effort for a representative Navy C4I project with 250 nominal requirements and 18
Apply a realistic Navy C4I profile for the cost driver ratings.

COSYSMO GPT

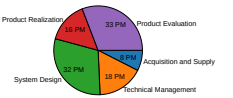
Input Category	Parameter	Value	Rationale / Assumption
System Size	System Requirements (Nominal)	250	Medium-scale C4I system with typical functional and performance requirements.
	System Requirements (Difficult)	180	Complex, high-criticality and interface-heavy requirements.
	Interfaces (Nominal)	40	Moderate integration with networks and external systems.
	Critical Algorithms (Nominal)	15	Includes moderate algorithmic complexity such as fusion, encryption, and routing.
	Operational Scenarios (Nominal)	20	Representative set of use cases across operational contexts.
Cost Drivers	Requirements Understanding	High	Stakeholders have strong domain knowledge and clear mission definition.
	Architecture Understanding	High	Assumed experienced with Navy reference architectures.
	Level of Service Requirements	High	Demands high availability, cybersecurity, and reliability.
	Migration Complexity	Nominal	Evolutionary upgrades to existing systems.
	Technology Risk	High	Emerging AI/ML, 5G, and cyber-resilient components.
	Documentation	Nominal	Baseline SE documentation per Navy standards.
	Diversity of Installations/Platforms	High	Deployed across shore, shipboard, and joint environments.
	Recursive Levels in Design	Nominal	Standard layered architecture, limited recursion.
	Stakeholder Team Cohesion	High	Strong Navy–contractor collaboration.
	Personnel/Team Capability	High	Experienced systems engineers with C4I expertise.
	Personnel Experience Continuity	Nominal	Some turnover expected over the program lifecycle.
	Process Capability	High	Mature systems engineering processes.
	Multisite Coordination	Nominal	Primarily co-located teams.
	Tool Support	Nominal	Standard SE tools used.
Other	AI Usage	Nominal	Some automation support, not a primary project driver.
Labor Rate	Cost per PM	$20,000	Typical DoD/Navy SE labor cost.

Estimate

Effort	107.9 Person-Months
Schedule	7.0 Months
Cost	$2,157,130
Total Size	1611.5 Equivalent Nominal Requirements

Phase / Activity	Conceptualize	Develop	Operational Test	Transition
Acquisition and Supply	1.1	2.2	0.5	0.3
Technical Management	5.4	9.7	6.5	4.3
System Design	12.9	15.1	6.5	3.2
Product Realization	2.7	6.5	6.5	5.4
Product Evaluation	7.5	10.8	16.2	6.5

Effort by Activity



Calculation Source: View on softwarecost.org

COSYSMO Tool Endpoint with AI Inputs

Constructive Systems Engineering Cost Model (COSYSMO)

This version of COSYSMO incorporates the new cost factor *AI Usage* with provisional effort multipliers. Setting *AI Usage* to Very Low produces equivalent estimates to COSYSMO versions 1 and 2 when other inputs are the same.

System Size

	Easy	Nominal	Difficult
System Requirements	0	250	180
System Interfaces	0	40	0
Critical Algorithms	0	15	0
Operational_Scenarios	0	20	0

Cost Factors

Requirements Understanding	High	Documentation	Nominal	AI Usage	Nominal
Architecture Understanding	High	Diversity of Installations/Platforms	High	Personnel Experience/Continuity	Nominal
Level of Service Requirements	High	Recursive Levels in Design	Nominal	Process Capability	High
Migration Complexity	Nominal	Stakeholder Team Cohesion	High	Multisite Coordination	Nominal
Technology Risk	High	Personnel/Team Capability	High	Tool Support	Nominal

Labor Rate

Cost per Person-Month (Dollars) 20000

Calculate

Results
Effort = 107.9 Person-Months
Schedule = 7.0 Months
Cost = $2,157,130
Total Size = 1611.5 Equivalent Nominal Requirements

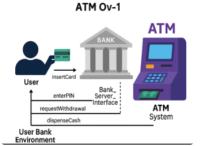
Effort Distribution (Person-Months)

Phase / Activity	Conceptualize	Develop	Operational Test and Evaluation	Transition to Operation
Acquisition and Supply	1.1	2.2	0.5	0.3
Technical Management	5.4	9.7	6.5	4.3
System Design	12.9	15.1	6.5	3.2
Product Realization	2.7	6.5	6.5	5.4
Product Evaluation	7.5	10.8	16.2	6.5



COSYSMO Tool

Example Cost Estimate 2



estimate the systems engineering effort for this system

Excellent — this ATM system operational view (OV-1) is a classic example for a **small, embedded financial transaction system** that interfaces with a bank backend.

Let's create a **COSYSMO-based estimation** for this ATM system.

🧩 Step 1: Interpret the System Context

From the OV-1:

- The **ATM System** includes both hardware and software components.
- The **Bank Server Interface** is an external system connection.
- The **User** interacts with the system through simple operations: card insertion, PIN entry, withdrawal request, and cash dispense.

This suggests a **moderate system size** with **limited number of interfaces and scenarios** — typical of a small-scale financial control system.

...

🧱 Step 2: Define System Size Inputs

Category	Parameter	Value	Rationale / Assumption
System Size	System Requirements (Nominal)	80	Basic functional/performance requirements (authentication, transaction, error handling).
	Interfaces (Nominal)	5	ATM ↔ Bank Server, ATM ↔ User I/O, ATM ↔ Cash Dispenser, Card Reader, Network Gateway.
	Critical Algorithms (Nominal)	3	Includes encryption/decryption, PIN validation, transaction verification.
	Operational Scenarios (Nominal)	5	Insert Card, Enter PIN, Withdraw Cash, Balance Inquiry, Transaction Cancel.

⚙️ Step 3: Cost Driver Inputs (defaulting to *Nominal* unless noted)

Category	Parameter	Value	Rationale / Assumption
Cost Drivers	Requirements Understanding	High	Clear financial domain and well-defined regulatory requirements.
	Architecture Understanding	High	Standard ATM architecture (modular with secure transaction channel).
	Level of Service Requirements	High	Requires high security, reliability, and availability.
	Migration Complexity	Low	New system or upgrade of existing model; low legacy constraints.
	Technology Risk	Nominal	Uses mature hardware and software technologies.
	Documentation	Nominal	Standard system documentation expected.

Local LLM and Tool Example I

```
r@mac:~/Documents/lmstudio> python lms_cosysmo_shell2_mdv_viewer.py

Local COSYSMO + LM Studio Shell
--------------------------------------------------------------------------------
  Component                      Configuration
    LLM Model                    |   mistral-medium-3.5-128b@q2_k
    LM Studio API                |   http://localhost:1234/v1
    COSYSMO Tool                 |   http://localhost:5555/cgi-bin/cosysmo.py
    Markdown Renderer            |   mdv
    Deterministic Safety Routing   Enabled
--------------------------------------------------------------------------------

 Commands
    - state  display the current COSYSMO scenario
    - reset  reset all scenario values to deterministic defaults
    - exit or quit  end the session

You: what is effort for 250 system requirements

LLM Thinking...
[ROUTER] COSYSMO calculation required.
[ROUTER] Forcing estimateCosysmo with tool_choice='required'.
[LLM] finish_reason = tool_calls
[LLM] tool_calls = 1
[TOOL] requested function = estimateCosysmo
[TOOL] raw arguments = {"System_Requirements_Nominal":250,"calculated":"True"}
[TOOL] Executing local deterministic COSYSMO tool.

COSYSMO Inputs
--------------------------------------------------------------------------------
  Input Category  Parameter                              Value       Rationale / Assumption
  System Size     System Requirements (Easy)             0           Unspecified size category; deterministic default = 0.
  System Size     System Requirements (Nominal)          250         Interpreted from current user request.
  System Size     System Requirements (Difficult)        0           Unspecified size category; deterministic default = 0.
  System Size     Interfaces (Nominal)                   0           Unspecified size category; deterministic default = 0.
  System Size     Interfaces (Difficult)                 0           Unspecified size category; deterministic default = 0.
  System Size     Critical Algorithms (Easy)             0           Unspecified size category; deterministic default = 0.
  System Size     Critical Algorithms (Nominal)          0           Unspecified size category; deterministic default = 0.
...
```

Local LLM and Tool Example II

```
 Cost Driver     AI Usage                                Nominal     Unspecified cost driver; deterministic default = Nominal.
 Cost Driver     Requirements Understanding              Nominal     Unspecified cost driver; deterministic default = Nominal.
 Cost Driver     Architecture Understanding              Nominal     Unspecified cost driver; deterministic default = Nominal.
 Cost Driver     Level of Service Requirements           Nominal     Unspecified cost driver; deterministic default = Nominal.
 Cost Driver     Migration Complexity                    Nominal     Unspecified cost driver; deterministic default = Nominal.
 Cost Driver     Technology Risk                         Nominal     Unspecified cost driver; deterministic default = Nominal.
 Cost Driver     Documentation                           Nominal     Unspecified cost driver; deterministic default = Nominal.
 Cost Driver     Diversity of Installations/Platforms    Nominal     Unspecified cost driver; deterministic default = Nominal.
 Cost Driver     Recursive Levels in Design              Nominal     Unspecified cost driver; deterministic default = Nominal.
 Cost Driver     Stakeholder Team Cohesion               Nominal     Unspecified cost driver; deterministic default = Nominal.
 Cost Driver     Personnel/Team Capability               Nominal     Unspecified cost driver; deterministic default = Nominal.
 Cost Driver     Personnel Experience/Continuity         Nominal     Unspecified cost driver; deterministic default = Nominal.
 Cost Driver     Process Capability                      Nominal     Unspecified cost driver; deterministic default = Nominal.
 Cost Driver     Multisite Coordination                  Nominal     Unspecified cost driver; deterministic default = Nominal.
 Cost Driver     Tool Support                            Nominal     Unspecified cost driver; deterministic default = Nominal.
 Labor Rate      Cost per Person-Month                   $20,000.00  Unspecified labor rate; deterministic default = $20,000 per
 ↪  Person-Month.
--------------------------------------------------------------------------------

[TOOL] Calling deterministic COSYSMO CGI...

COSYSMO Results
    - Effort: 88.4 PM
    - Schedule: 6.6 Months
    - Cost: $1,768,805.00
    - Total Size: 250 Equivalent Nominal Requirements

 Calculation Source: View on local COSYSMO tool
 [1] http://localhost:5555/cgi-bin/cosysmo.py?System_Requirements_Easy=0&System_Requirements_Nominal=250&System_Requirements_
↪   Difficult=0&Interfaces_Easy=0&Interfaces_Nominal=0&Interfaces_Difficult=0&Critical_Algorithms_Easy=0&Critical_Algorithms
↪   _Nominal=0&Critical_Algorithms_Difficult=0&Operational_Scenarios_Easy=0&Operational_Scenarios_Nominal=0&Operational_Scen
↪   arios_Difficult=0&AI_Usage=Nominal&Requirements_Understanding=Nominal&Architecture_Understanding=Nominal&Level_of_Servic
↪   e_Requirements=Nominal&Migration_Complexity=Nominal&Technology_Risk=Nominal&Documentation=Nominal&Diversity_of_Installat
↪   ions_Platforms=Nominal&Recursive_Levels_in_Design=Nominal&Stakeholder_Team_Cohesion=Nominal&Personnel_Team_Capability=No
↪   minal&Personnel_Experience_Continuity=Nominal&Process_Capability=Nominal&Multisite_Coordination=Nominal&Tool_Support=Nom
↪   inal&system_labor_cost_per_PM=20000&calculated=True
```

Live Demonstrations

Verification and Governance Principles

- Maintain computational model versions, calibration or analysis bases, API schemas, assistant instructions, and knowledge artifacts under configuration control.
- Test equivalence classes and boundary conditions for delegated quantitative services, compare API outputs against reference tool runs.
- Verify parameter normalization, default/assumption behavior, request construction, output parsing, and provenance presentation independently from numerical model verification.
- Log interpreted inputs, assumptions, tool requests, model versions, and returned results for audit and regression testing.
- Treat prompt instructions, schemas, knowledge bases, and tool interfaces as controlled engineering artifacts within the digital thread.

Results and Discussion

- Across system reliability and cost examples, deterministic delegation produced reproducible backend tool outputs while preserving natural conversational interaction.
- User requests were translated into complete, reviewable, quantitative parameter sets.
- Authoritative computation remained within validated external tools, so hallucination risk is bounded to interpretation and elicitation rather than the numerical computation itself.
- Smart input inference reduces modeling burden, including complex narrative to model transformations that otherwise require specialized domain and tool familiarity.
- Structured provenance, exposed assumptions, source URLs, and detailed outputs improve transparency, auditability, regression testing, and organizational trust.

Conclusions

- Deterministic delegation provides a practical pattern for trustworthy AI in quantitative systems engineering.
- LLM is constrained to interpretation, clarification, assumption management, orchestration, and presentation while authoritative computation remains in verified external tools.
- System reliability and cost modeling examples span increasing modeling complexity, from direct scalar calculations to structured model construction from narrative descriptions.
- Smart multi-modal input inference lowers barriers to quantitative methods while reviewable inputs and provenance preserve engineering oversight.
- Benefits extend beyond computational correctness to reproducibility, auditability, governance, regression testing, digital-thread interoperability, and reduced user cognitive burden.
- The pattern generalizes to other LLM platforms and quantitative systems engineering domains without sacrificing determinism, traceability, transparency, or auditability.

Future Work

- Extend deterministic delegation to additional quantitative systems engineering applications.
- Add AI input assistance directly integrated with computational tools via LLM APIs, eliminating need for separate GPTs.
- Consolidate separate system reliability capabilities into a more general-purpose reliability assessment environment with an AI front end.
- Add AI assisted project risk assessment in conjunction with cost estimation [9].
- Expand multi-modal smart input interpretation with more robust instructions and domain knowledge.
- Apply formal automated verification across structured input equivalence partitions and regression-test delegated computations against reference results.
- Evaluate with practitioners in real-world project settings and integrate with broader digital engineering environments and LLM platforms.

References I

[1] R. Madachy. "Trustworthy AI for Quantitative Systems Engineering via Deterministic Delegation to Verified Tools". In: *INCOSE Insight* (2026). URL: https://incose.onlinelibrary.wiley.com/doi/10.1002/inst.70056.

[2] R. Valerdi. "The Constructive Systems Engineering Cost Model (COSYSMO)". PhD thesis. Los Angeles, CA: University of Southern California, 2005.

[3] R. Madachy. *Parallel Component Reliability Analyzer GPT*. https://chatgpt.com/g/g-689e7a63ef1c81919c636501c6bfac4b-system-reliability-service. 2025.

[4] R. Madachy. *Parallel Component Reliability Analyzer*. https://se-lib.org/online/parallel_component_reliability.py. 2026.

[5] R. Madachy. *Reliability Block Diagram Expert GPT*. https://chatgpt.com/g/g-69cac3ce7fb08191a6717276820fa2c2-reliability-block-diagram-expert. 2026.

[6] R. Madachy. *Reliability Block Diagram Analyzer*. https://se-lib.org/online/rbd.py. 2025.

[7] R. Madachy. *COSYSMO GPT*. https://chatgpt.com/g/g-68a4f33c568881919f1edf346af27e0b-cosysmo-gpt. 2025.

References II

[8] R. Madachy. *COSYSMO Tool with AI Usage*. http://softwarecost.org/tools/COSYSMO_AI. 2025.

[9] R. Madachy and R. Valerdi. "Expert COSYSMO Systems Engineering Cost Model and Risk Advisor". In: *Handbook of Military and Defense Operations Research*. Ed. by N. Scala and J. Howard. Boca Raton, FL: CRC Press, 2020.